

SkyNYC

Measuring weather impact on New York airport arrivals
from raw aircraft telemetry

Final report

Gil Lu

August 2026

Abstract

SkyNYC derived airport operational events — arrivals, holding patterns, and go-arounds — for JFK, LaGuardia, and Newark directly from raw ADS-B position telemetry, validated them daily against an independent arrivals record, and joined them with official weather observations to measure how conditions degrade arrival operations. No delay feed, arrivals feed, or event feed was consumed at any point: every operational event was computed by per-aircraft state machines over a Kafka-buffered position stream, and the accuracy of that derivation was published as a daily precision/recall score rather than claimed. Over a fifteen-day continuous run the detectors identified 26,303 arrivals, scored against 28,331 independently recorded ones: mean daily precision 96.7%, mean daily recall 93.0% across the three airports. A companion historical layer rebuilt 181 million federal on-time records (1987–2026) into a Delta Lake medallion, quantifying the thesis at climatological scale: average arrival delay on the worst weather days runs eight to fourteen times higher than on clear days. Total infrastructure cost: about \$26 cash and a small fraction of a \$200 cloud credit grant.

1 The question and the approach

The business question: how much do arrival rates fall — and holding and go-arounds rise — as conditions degrade from VFR to IFR, and which weather variable (visibility, ceiling, wind and gusts) leads the degradation?

The engineering premise that makes the project interesting: the operational events needed to answer it exist in no public feed. The FAA publishes delay status, not events; airlines publish schedules, not go-arounds. SkyNYC therefore treats event *derivation* as the core deliverable — and holds that derivation to a published accuracy standard, scored daily against ground truth that the detectors never see.

The system ran in two layers:

- **Live streaming layer** (2026-08-12 to 2026-08-26): OpenSky Network ADS-B state vectors polled every 30s and NOAA/NWS observations and alerts polled every 5/2 minutes, buffered in Kafka, processed by Spark Structured Streaming into a bronze Parquet archive on Azure ADLS Gen2, a live-position serving table, and a stream of derived flight events in Postgres. dbt built hourly marts on top; Dagster orchestrated the batch edges (daily ground-truth pull, hourly builds, nightly backup).

- **Historical batch layer:** 38 years of BTS on-time performance (181M rows) and 4.3M METAR observations, landed by Azure Data Factory and built into a bronze/silver/gold Delta medallion by Databricks jobs that Dagster triggers over the Jobs API. Postgres remained the only serving store.

2 Architecture

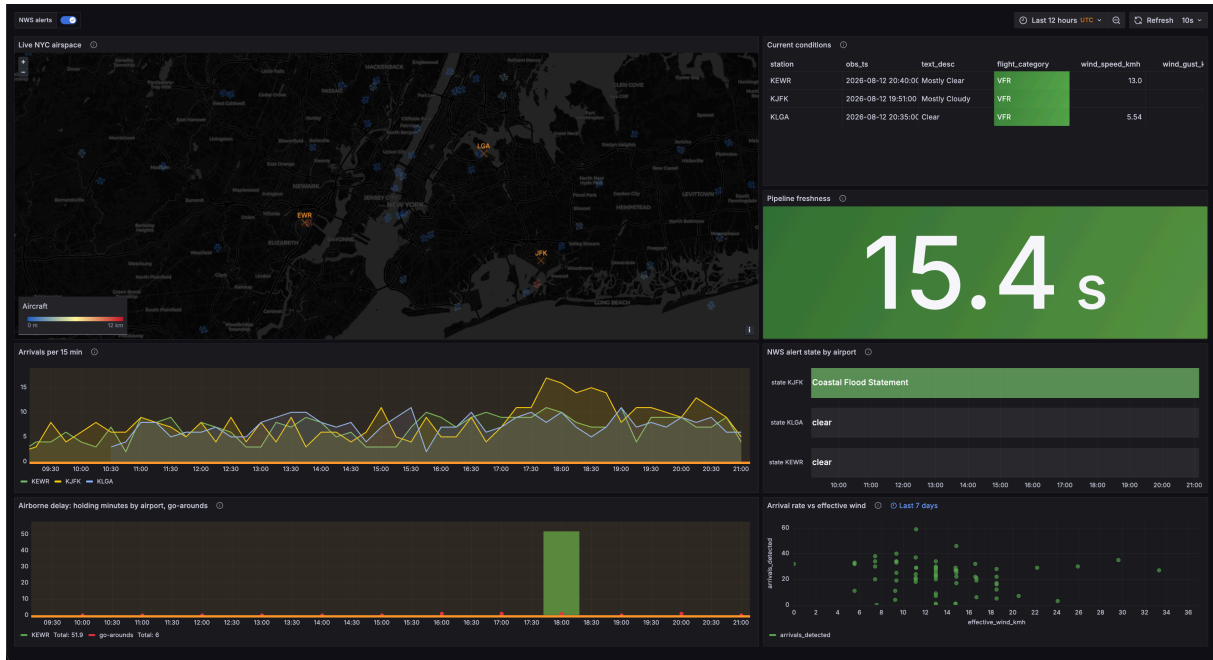


Figure 1: The live operations dashboard during the run: heading-rotated aircraft colored by altitude, per-station flight category, pipeline freshness, detected arrivals per 15 minutes with NWS alert windows overlaid.

Nine containers on one DigitalOcean droplet (8 GB, Docker Compose): Kafka (KRaft, single node), Postgres 16, Spark (single-node local[*], three checkpointed streaming queries), two Python producers, a weather consumer, Grafana, and the Dagster webserver and daemon. Azure held what must survive the droplet: the immutable bronze archive and database backups.

Design decisions that carried the project:

- **Kafka at 5 messages per second** — replay is the feature, not throughput. The position API serves at most one hour of history, so the retained log (7 days) is the only substrate detector tuning has. Both production incidents were healed from it without data loss.
- **Docker supervises streams; Dagster supervises batch.** An orchestrator’s run model is the wrong shape for an infinite process.
- **Join-at-read, not stream-stream** — weather is three slowly-changing station records; events are enriched at query time with the observation valid at event time.
- **Units and nulls are load-bearing.** Every NWS quantity converts by reading its `unitCode` (wind arrives in km/h); position-less telemetry is counted and dropped, never

interpolated; on-ground aircraft legitimately report null altitude.

- **Idempotency everywhere** — deterministic event ids and upserts on natural keys make replays and crash recovery unable to duplicate rows.

3 Event detection

Per-aircraft state machines over a ring buffer of recent position samples, keyed by transponder address; Kafka partitioning guarantees per-aircraft ordering into Spark's `applyInPandasWithState`.

- **Arrival** — approach candidacy (near field, low, descending) confirmed by ground contact, or by coverage loss on short final: transponder contact commonly drops below ~ 450 m on approach, so a disappearance there, confirmed by a 90-second state timeout, is an arrival.
- **Holding** — at least 340° of cumulative unwrapped track change within eight minutes without net displacement, inside the holding altitude/speed band, filtered to airline-class traffic.
- **Go-around** — descent on approach followed by a committed climb (two consecutive strong-climb samples, at least 200 m regained) with no touchdown.

Every threshold is a named constant. Tuning was done by replaying the retained Kafka log with adjusted values and re-scoring against ground truth — never by eyeballing the map. The detectors have no Spark dependency; the test suite runs the exact production logic over recorded live sequences, including the captured poison sequence from the August 14 incident (section 7).

4 Validation

Every morning Dagster pulled the previous day's arrivals for the three airports from OpenSky's independent arrivals record — a source the detectors never see — and dbt scored each day per airport: a detected arrival matches ground truth when the same airframe arrives at the same airport within ten minutes. Precision is matched detected over all detected; recall is matched ground truth over all ground truth.

The mart publishes both figures structurally, not on trust:

- Precision publishes only once the day's ground truth has landed at plausible volume (at least half the detector's own count — the upstream aggregation can lag days).
- Recall additionally requires the detection stream to have observed at least 20 of 24 hours, measured stream-wide: an hour counts as observed when any of the three fields recorded an arrival. A quiet overnight hour at one airport is traffic, not coverage loss; a true outage silences all three fields at once and still withholds the score.
- An unscored day renders as a gap — never as a zero. A partial day and a bad day must never look alike.

5 Results: the live run

Fourteen days scored end to end (2026-08-12 through 2026-08-25; the first partial day published precision only, and the final day of operation is unscored because its ground truth publishes the

following morning). Targets: precision $\geq 85\%$, recall $\geq 80\%$, per airport per day.

Airport	Scored days	Mean P	Min P	Mean R	Min R
JFK	14	96.1%	89.4%	97.6%	92.8%
LaGuardia	14	97.8%	89.0%	95.8%	90.0%
Newark	14	96.0%	87.0%	85.7%	76.3%

Table 1: Detection quality over the full run (fully scored days). The single target miss of the run is Newark’s recall on August 24 (76.3%), part of an across-the-board dip that day — every field’s precision and recall fell together, pointing at the day’s data rather than any one detector. Every other airport-day met both targets.

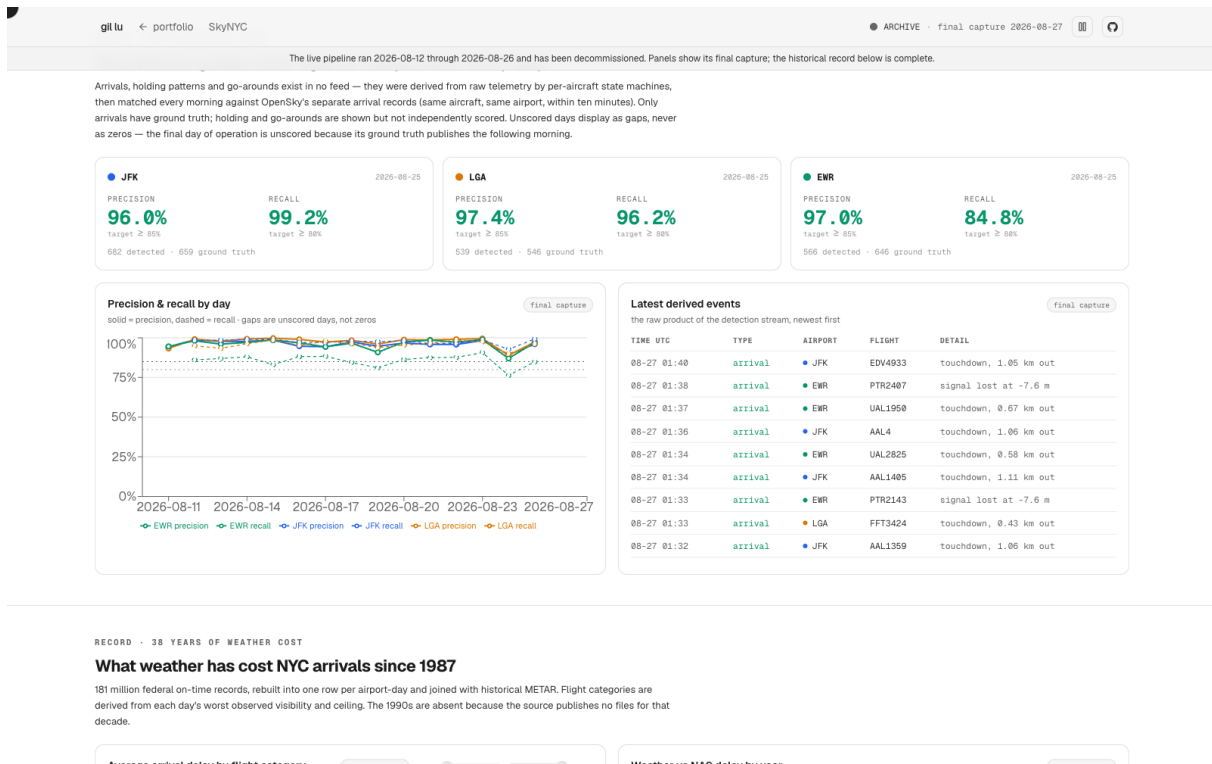


Figure 2: The public dashboard’s validation section at close: daily precision (solid) and recall (dashed) per airport against the $P \geq 85\%$ / $R \geq 80\%$ targets, and the final scored day’s tiles.

Newark’s recall ran consistently below the other two fields (76.3–90.6%, mean 85.7%). The pattern is structural, not noise: EWR sits at the edge of the receiver coverage the detectors work from, and its arrival flows more often descend below reliable coverage earlier in the approach, so more true arrivals produce too few samples to confirm. The precision guard means what the detector does claim at Newark is still right — the miss cost lands on recall alone, which is the honest side to pay it.

6 Results: 38 years of weather cost

The historical medallion quantifies the thesis at climatological scale. Table 2 aggregates 31,413 airport-days (1987–2026; BTS publishes no files for 1990–1999) by each day’s worst observed flight category.

Airport	Category	Avg. arrival delay (min)	Cancelled	Days
JFK	VFR	2.0	1.2%	5,428
	MVFR	7.5	2.1%	1,835
	IFR	16.1	3.5%	1,537
	LIFR	22.9	5.5%	1,669
LGA	VFR	2.1	2.1%	5,428
	MVFR	7.8	3.2%	1,952
	IFR	17.6	5.3%	1,686
	LIFR	29.7	9.7%	1,403
EWR	VFR	4.1	1.6%	5,305
	MVFR	10.7	2.6%	1,962
	IFR	21.9	4.7%	1,800
	LIFR	31.1	8.1%	1,402

Table 2: Average arrival delay and cancellation rate by the day’s worst observed flight category, 1987–2026. Delay climbs monotonically at every airport — roughly $11\times$ VFR-to-LIFR at JFK, $14\times$ at LaGuardia, $8\times$ at Newark — and cancellation rates climb four- to five-fold.

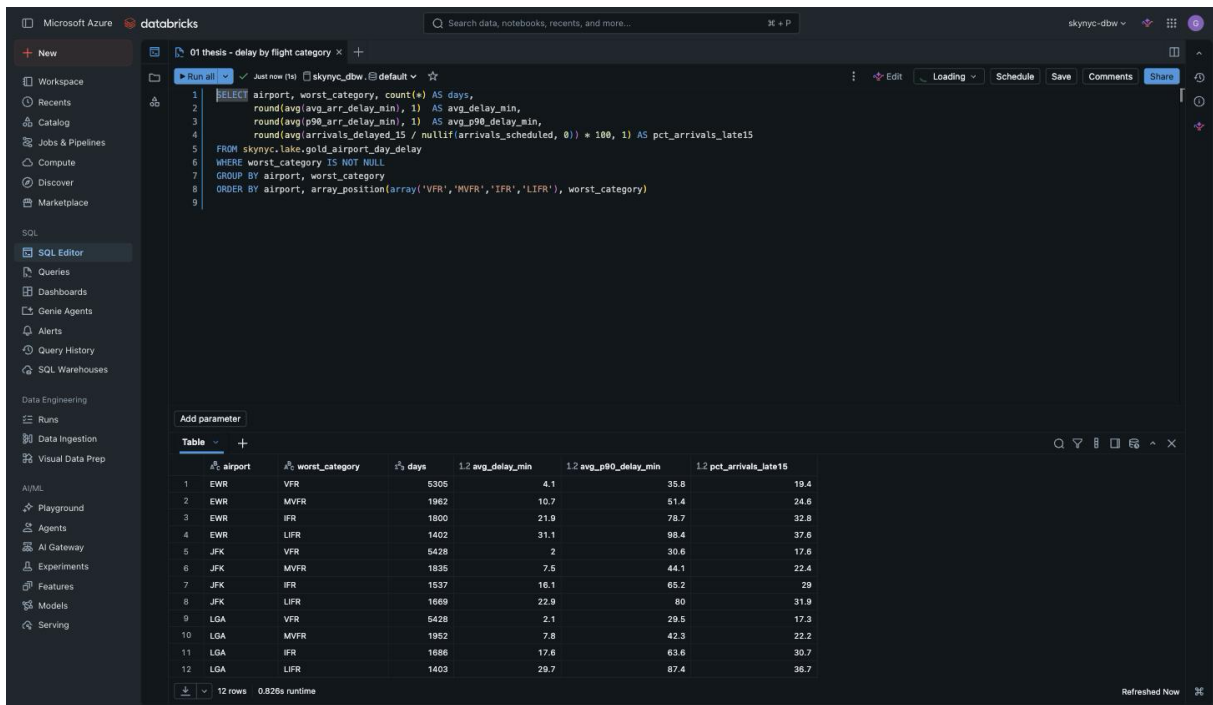


Figure 3: The thesis in one query on the serverless warehouse over the gold Delta tables, sub-second: arrival delay by flight category across the full record.

Visibility and ceiling — the components of flight category — are the strongest leading indicators in the record; wind and gusts matter most through runway-configuration changes, which show up as IFR-day delay variance rather than as an independent monotonic effect.

7 Operations

The live layer ran continuously from 2026-08-12 to 2026-08-26 on one droplet. Steady state: about 4.7 GB RAM across nine containers, three streaming queries each completing micro-batches well inside their trigger intervals, and an API credit budget (3,000 of 4,000 daily) never exhausted. Two incidents occurred; both were diagnosed from evidence, fixed at the root, and healed without data loss.

7.1 Incident 1: NaN poisoning (Aug 14–15)

The events streaming query entered a crash loop: pandas represents nullable doubles as NaN, which passes its `is None` guards, compares false against every threshold, and survives `json.dumps` as a literal NaN that Postgres’s `jsonb` rejects. One poisoned record made the micro-batch fail and replay from the checkpoint — 1,600+ restarts over 23 hours. The fix normalized NaN to `None` at both seams where the Spark bridge hands the detectors data: the message boundary and the decoded state blob (state written before the first fix retained the poison, which is why fixing only the message boundary did not recover the stream). The gap was replayed in full from the retained Kafka log; the poison sequence was captured into the test suite. Producers and the live path never went down.

7.2 Incident 2: silent scheduler stop (Aug 23)

Dagster’s scheduler daemon thread hung at 19:00 UTC evaluating an hourly schedule — no exception, no restart, container healthy, every other daemon thread still heart-beating. Scheduled work simply stopped: no more hourly builds, ground-truth pulls, or quality scoring. Detection itself was unaffected (streams are supervised by Docker, not Dagster — the separation held exactly as designed). The stop was diagnosed from the per-daemon heartbeat table; restarting the daemon let the scheduler’s catch-up fire the missed ground-truth partitions, and the idempotent upsert-everywhere design meant three days of scores backfilled cleanly with no special handling. Lesson recorded: container-level health checks do not see thread-level death; liveness monitoring must watch the heartbeat the component itself emits.

8 Cost

Item	Cost	Notes
DigitalOcean droplet	\$26	4 GB tier resized to 8 GB; hourly billing, 2026-08-11 to 2026-08-27, destroyed at close
Azure (ADLS, ADF, Databricks)	~\$10–15	against a \$200 credit grant, most of it the one-time medallion build
Vercel (frontend + blob)	\$0	free tier
Total cash	~\$26	against a \$150 budget cap

Table 3: Project infrastructure cost, complete.

9 Decommissioning

The project concluded 2026-08-26. Compute was destroyed; everything durable was preserved first:

- Final ground-truth pull, quality scoring, and mart build completed, so the record closes on a fully scored day (2026-08-25).
- Final `pg_dump` archived locally and off the droplet.
- The public dashboard (sky.gillu.me) now serves its final capture from static storage — the page states it is an archive, the map shows the final positions, and the historical sections remain complete. Outliving its own pipeline was a designed property, not an afterthought.
- The droplet and the Azure resource group (lake, Data Factory, Databricks workspace) were deleted; no billable resources remain.

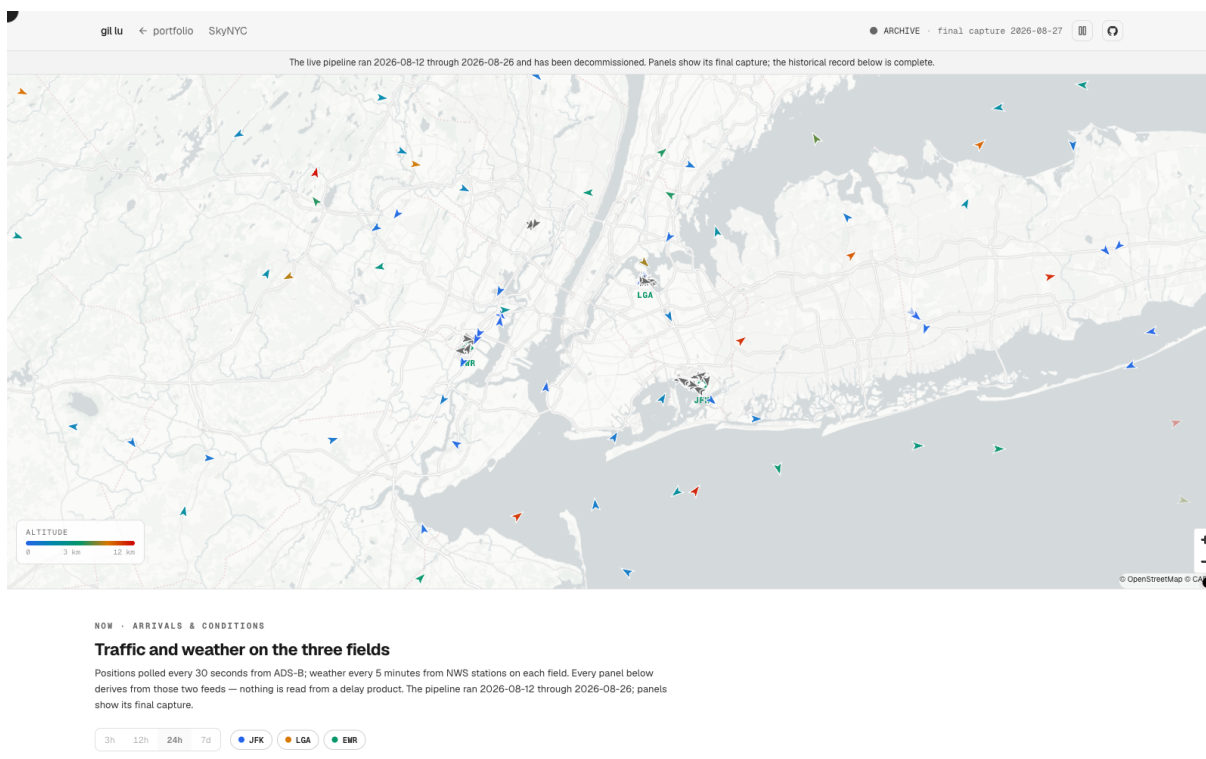


Figure 4: The public dashboard in its final state: the archive banner, the final captured positions, and per-field conditions at close.

Data sources

- **OpenSky Network** — aircraft state vectors and arrival records, research/non-commercial terms. M. Schäfer, M. Strohmeier, V. Lenders, I. Martinovic, M. Wilhelm. *Bringing Up OpenSky: A Large-scale ADS-B Sensor Network for Research*. IPSN 2014.
- **NOAA / National Weather Service** — station observations and active alerts, US-government open data.
- **U.S. BTS TranStats** — on-time performance, 1987–2026.
- **Iowa Environmental Mesonet** — historical METAR.

A Daily scores

Percent, as published by `mart_detection_quality`. A dash is a withheld score, not a zero: August 11 was a partial first day (five observed hours), so recall is withheld; August 10 predates detection entirely; August 26's ground truth publishes after decommissioning.

Date (2026)	JFK		LaGuardia		Newark	
	P	R	P	R	P	R
08-11	94.4	—	93.3	—	94.6	—
08-12	98.8	98.5	99.1	95.0	98.1	86.1
08-13	97.7	98.6	98.0	93.4	96.0	87.0
08-14	97.8	98.9	99.3	96.3	97.1	88.0
08-15	98.5	99.1	99.8	98.6	98.6	83.1
08-16	94.7	97.1	99.0	95.3	96.8	88.2
08-17	94.4	97.0	97.3	97.9	94.3	88.3
08-18	97.4	97.9	98.4	96.3	96.6	84.4
08-19	94.7	97.6	96.1	93.9	90.8	81.1
08-20	96.7	98.2	98.8	95.4	97.4	86.3
08-21	95.8	96.4	98.7	96.5	98.6	87.6
08-22	95.8	96.7	99.1	98.6	97.3	87.7
08-23	98.3	98.6	99.5	97.5	98.8	90.6
08-24	89.4	92.8	89.0	90.0	87.0	76.3
08-25	96.0	99.2	97.4	96.2	97.0	84.8

Table 4: Daily precision and recall per airport, full run.